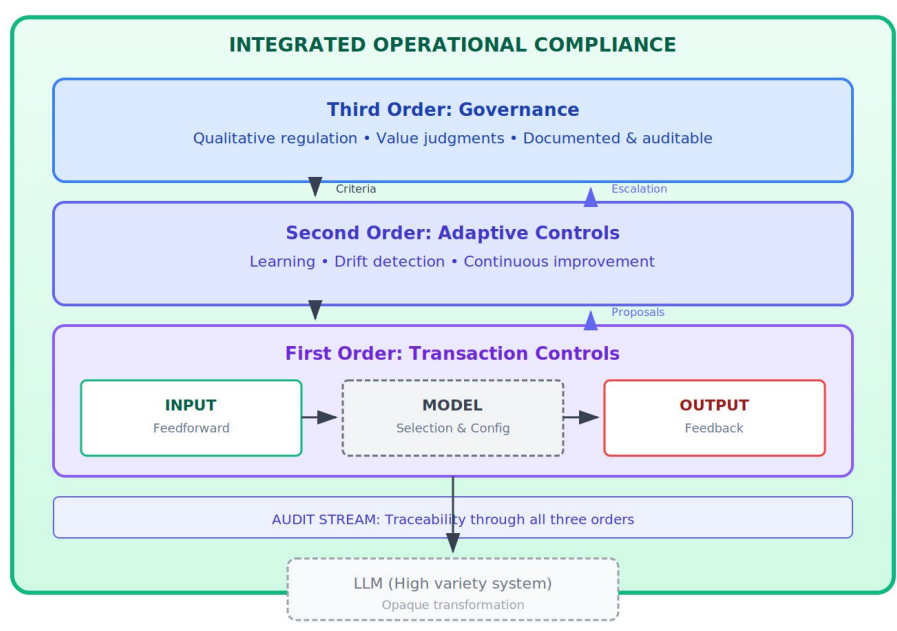


Governing Large Language Models

A Cybernetic Approach to AI Compliance

Governing Large Language Models

A Cybernetic Approach to AI Compliance



Based on Ashby's Law of Requisite Variety and Orders of Regulation

Raimund Laqua, P.Eng., PMP

Lean Compliance Consulting

November 2025

Executive Summary

Large Language Models present a fundamental governance challenge: how do you regulate a system whose internal workings are opaque and whose behavioral variety is effectively infinite? This briefing examines the technical architecture of LLMs and proposes an operational compliance framework grounded in cybernetic principles — specifically W. Ross Ashby's Law of Requisite Variety and Mick Ashby's work on ethical regulators.

The Lean Compliance operational compliance model is holistic — it encompasses governance as an integral layer, not as something separate that operational controls merely serve. The architecture described here includes all three orders of regulation: the transaction controls that act on each interaction, the adaptive controls that keep those controls effective, and the governance layer where value judgments are made.

The core insight is that effective LLM regulation requires controlling what you can — inputs, model selection, and outputs — while accepting that the transformation between input and output remains a black box. R. Ashby's Law tells us that static controls will always be insufficient against a high-variety system; the operational compliance architecture must itself be capable of learning and adaptation.

But there is a deeper insight from M. Ashby's work: regulation exists in orders, and only at the third order do we reach qualitative — that is, ethical — regulation. First and second order regulation is technical: controls acting on transactions, and controls ensuring those controls remain effective. Third-order regulation involves value judgments about what outcomes *should* be permitted. In an integrated operational compliance model, all three orders are integrated — governance is the third-order layer within the system, subject to the same discipline of documentation, traceability, and continuous improvement.

This integration is not merely organizational. It recognizes that governance decisions — the criteria by which compliance is judged — must themselves be governed: documented, justified, reviewable, and auditable.

What You're Actually Governing

When you download or deploy an LLM, you're working with a mathematical function encoded as numerical weights — billions of parameters that define how input tokens map to output probability distributions. Understanding what is and isn't controllable is essential to sound governance.

What's in the model

The weights encode everything the model "knows" — its knowledge, reasoning patterns, biases, safety behaviors, and capabilities. These emerged from training on massive datasets through processes that are rarely fully disclosed. You cannot inspect the weights to determine what the model will do; you can only observe behavior empirically.

The model also requires a specific prompt architecture — special tokens and formatting that the model expects based on its training. Using incorrect formatting degrades performance unpredictably. This architecture varies between model families with no universal standard.

What's outside the model

Common augmentation patterns like Retrieval-Augmented Generation (RAG) operate entirely outside the model. RAG retrieves relevant documents and prepends them to the prompt — the model itself is unchanged. It simply receives a longer input. This distinction matters for governance because RAG introduces a separate set of controls and failure modes unrelated to model behavior.

Similarly, when you provide a system prompt instructing the model to behave in certain ways, you haven't configured the model. You've provided input tokens that statistically correlate with desired output patterns. The model has no mechanism to guarantee it will follow instructions — it's always just predicting the next most probable token.

The execution layer

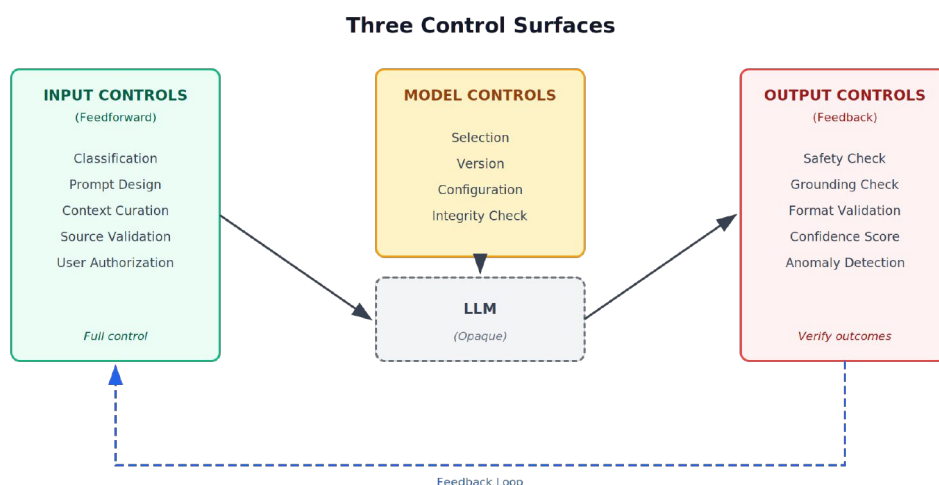
The inference runtime (Ollama, llama.cpp, vLLM, etc.) executes the mathematical operations defined by the weights. While implementations differ in how they optimize computation, the underlying mathematics is invariant. Given identical weights, architecture, and input, any correct implementation produces the same probability distributions over possible next tokens.

However, the output you actually receive depends on how the runtime *samples* from that distribution. With temperature greater than zero — which is the typical default — randomness is introduced. The same input to the same model can produce different outputs on successive runs. Even with identical prompts, you may get different responses.

This has direct governance implications. You cannot verify model behavior by testing a prompt once; the output is a sample from a distribution, not a deterministic result. Verification must be statistical: run enough samples to characterize the distribution of possible outputs. And even then, low-probability outputs — rare but possible — may not appear in your test samples.

The Three Control Surfaces

Operational compliance for LLMs means verifying, on each transaction, that the system's behavior falls within defined bounds. Since you cannot govern what happens inside the model, effective operational compliance requires controlling the three surfaces you can influence: inputs, model selection, and outputs.



Input controls (feedforward)

This is where you have the most leverage. Input controls shape conditions before inference to make desired outcomes more probable.

Feedforward control is proactive. You're not waiting to see what goes wrong; you're designing the input space to drive toward success. This includes classifying incoming requests to route or reject them appropriately, curating context through validated sources, designing prompts that establish behavioral expectations, and validating that all information provided to the model has known provenance.

Strong feedforward control reduces the burden on output validation. Every failure caught at the output stage represents a cost — compute, latency, user experience, and potential harm if the validator misses something.

Model controls

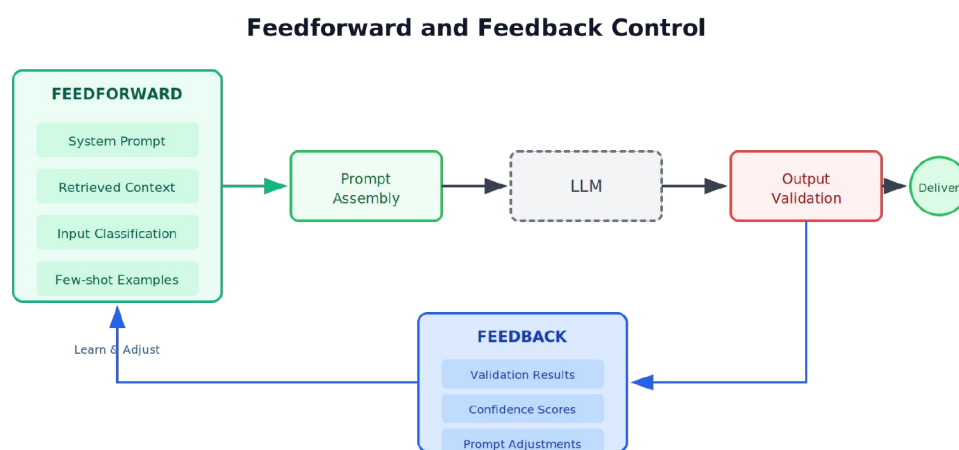
You can't modify the weights, but you control which model runs, which version, which configuration parameters, and whether it passes integrity checks before deployment.

This includes maintaining a registry of approved models, validating model file integrity against known hashes, controlling inference parameters like temperature and token limits, and ensuring deterministic settings where reproducibility is required.

Output controls (feedback)

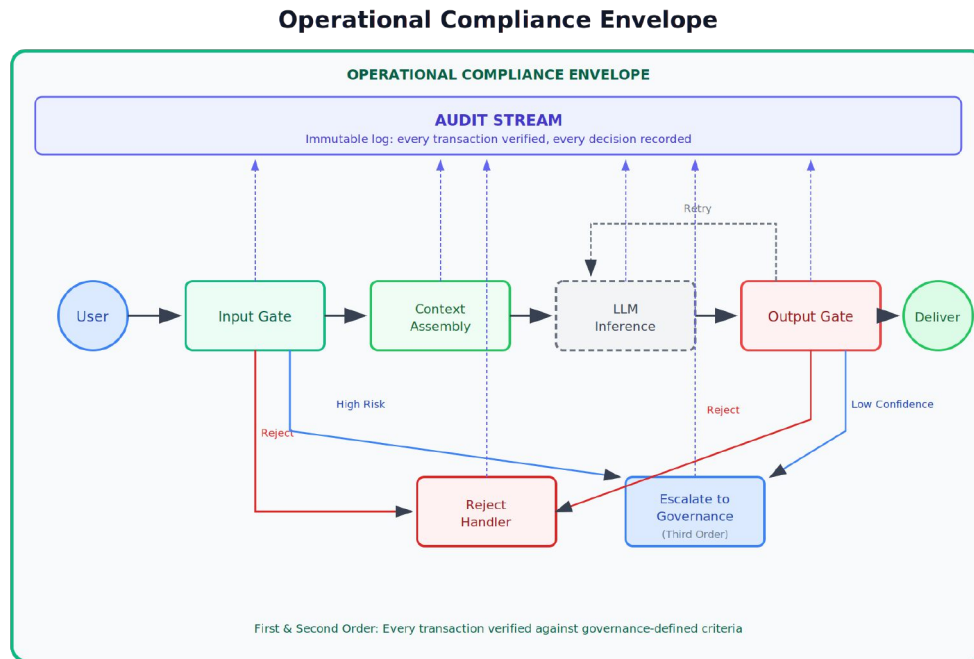
Output validation catches what input controls failed to prevent. This includes safety validation to verify outputs meet defined standards, grounding checks to ensure responses are based on provided context, format validation to confirm outputs meet structural requirements, and confidence assessment to identify uncertain or potentially problematic outputs.

Feedback control is reactive but essential. It provides the verification that promises were kept and the data for improving feedforward controls.



The Operational Compliance Envelope

The three control surfaces operate within an operational compliance envelope that provides structure, auditability, and escalation paths.



The envelope wraps all system activity in an audit stream — an immutable log of inputs, outputs, decisions, and metadata. Every gate, every routing decision, every validation result is recorded. This addresses the traceability requirement in regulations like the EU AI Act, which mandates that high-risk AI systems "technically allow for the automatic recording of events (logs) over the lifetime of the system" to ensure traceability appropriate to the system's intended purpose. The audit stream enables you to demonstrate, after the fact, that operational compliance was maintained — or to identify exactly where and how it wasn't.

The envelope also defines explicit paths for rejection and escalation. When input classification identifies high-risk requests, when output validation fails, when confidence scores fall below thresholds — the system has defined routes for these cases rather than undefined behavior. This is operational compliance in action: every transaction either completes within bounds or is handled through a defined exception path.

Ashby's Law and Its Implications

Here we encounter a fundamental constraint articulated by W. Ross Ashby: the Law of Requisite Variety. A regulator can only control a system if it has at least as much variety — as many possible states and responses — as the system being regulated.

The variety mismatch

LLMs have enormous variety. The input space is effectively infinite. The output space is similarly vast. The mapping between them is complex, nonlinear, and learned from data we cannot fully inspect.

A static, rule-based control system has limited variety. It can only recognize categories it was designed to detect and respond in ways it was programmed to respond. Against a high-variety system, static controls will always have blind spots.

This isn't a matter of building better rules. It's a mathematical constraint. You cannot achieve effective regulation with a regulator of insufficient variety.

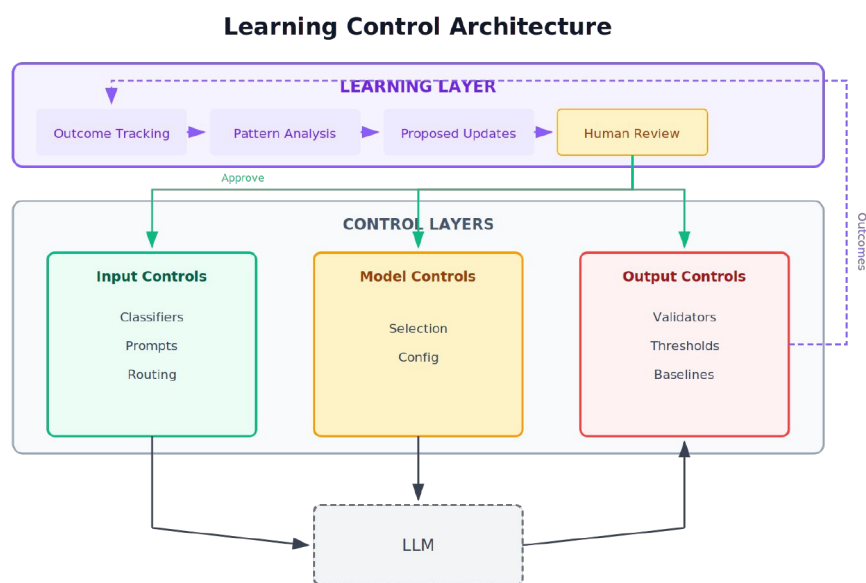
The isomorphism requirement

Your control architecture must mirror the complexity of what it's controlling. If the LLM processes natural language, your controls must understand natural language. If the LLM produces context-dependent behavior, your controls must reason about context. If the LLM can generate novel outputs, your controls must generalize to cases they haven't seen.

This leads to an important conclusion: your validators, classifiers, and anomaly detectors likely need to be models themselves. Only systems with similar representational capacity can match the variety of what they're regulating.

Learning controls

Static controls become stale. The threat landscape evolves. Users find new edge cases. The variety mismatch grows over time unless controls adapt.

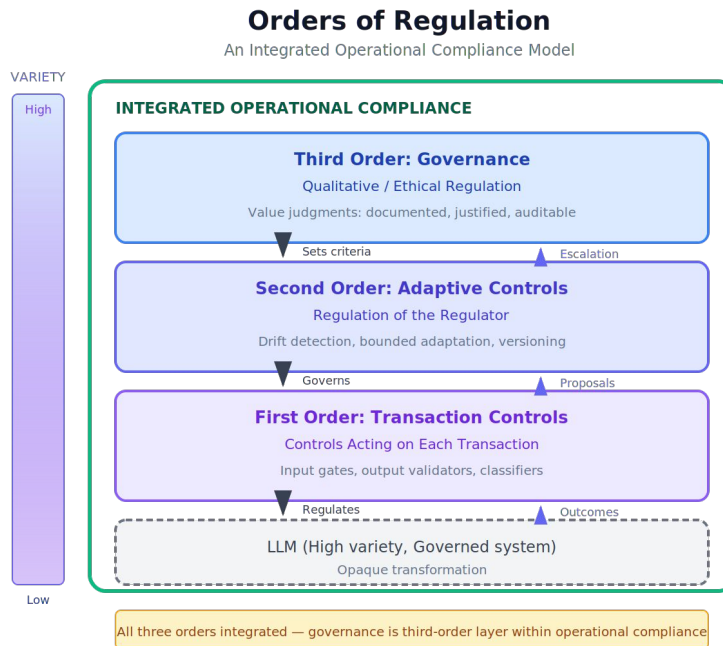


A governance architecture that learns might include classifiers that improve at detecting risk categories, prompt templates that evolve based on outcome data, validators that learn to detect new failure modes, and confidence estimators that calibrate based on actual outcomes.

The system identifies patterns; the learning layer proposes control updates; human review authorizes changes; deployed controls improve over time. This maintains requisite variety while keeping humans in the loop for authorization.

Orders of Regulation Within Operational Compliance

Ashby's work goes beyond the Law of Requisite Variety to describe orders of regulation — a hierarchy that, in an integrated operational compliance model, forms integrated layers of a single system.



First-order regulation

The regulator itself — the mechanism that acts on the system. In our architecture, this is the input gates, output validators, and classifiers. They take actions: permit, reject, route, retry.

This is operational compliance at the transaction level. Each request through the system is evaluated against defined criteria. The first-order controls are the machinery that makes or breaks compliance on each interaction.

Second-order regulation

Regulation of the regulator — governing how the first-order controls operate. This is our learning layer and meta-controls: drift detection, bounded adaptation, threshold adjustment, versioning. Second-order regulation ensures the first-order controls remain effective as conditions change.

This is operational compliance ensuring the compliance machinery itself stays fit for purpose. It's the difference between checking each transaction and checking that your checking is still valid.

Third-order regulation

Regulation of the second-order regulator. This is where Ashby identified qualitative regulation emerges — because at this level, you are no longer asking "does this output pass the check?" or even "is this check working properly?" You are asking "is this the right check to be running?"

That is a value judgment. It requires reasoning about purposes, about what outcomes are desirable, about tradeoffs between competing goods. Third-order regulation is ethical regulation — and in an integrated model, it is the governance layer within operational compliance.

Governance as integrated third-order regulation

In the Lean Compliance model, governance is not external to operational compliance — it is the third-order layer within it. This integration has important implications:

Governance decisions are subject to compliance discipline. The criteria by which first and second order controls operate must themselves be documented, justified, versioned, and auditable. Value judgments don't escape the rigor applied to technical controls.

The compliance streams flow through all three orders. The escalation stream doesn't hand off to something outside the system — it routes to the governance layer within the system. Human review is part of the operational flow, not an external override.

Continuous improvement applies to governance too. Just as first-order controls learn from outcomes and second-order controls adapt to maintain effectiveness, third-order governance must examine whether its criteria remain appropriate to evolving purposes and contexts.

The "human review" node in our architecture is not a boundary where operational compliance ends. It is the location of third-order regulation within the system — where values are applied, where criteria are examined, where purposes are reconsidered. The humans making these decisions are part of the operational compliance system, accountable to the same standards of documentation and traceability.

Alignment with Current AI Governance Frameworks

This cybernetic framing aligns well with where AI governance is actually landing in regulatory and standards bodies.

EU AI Act — Human oversight isn't optional or transitional; it's structural. High-risk systems require human authority over the system's operation. The regulation assumes human governance is the permanent third-order layer.

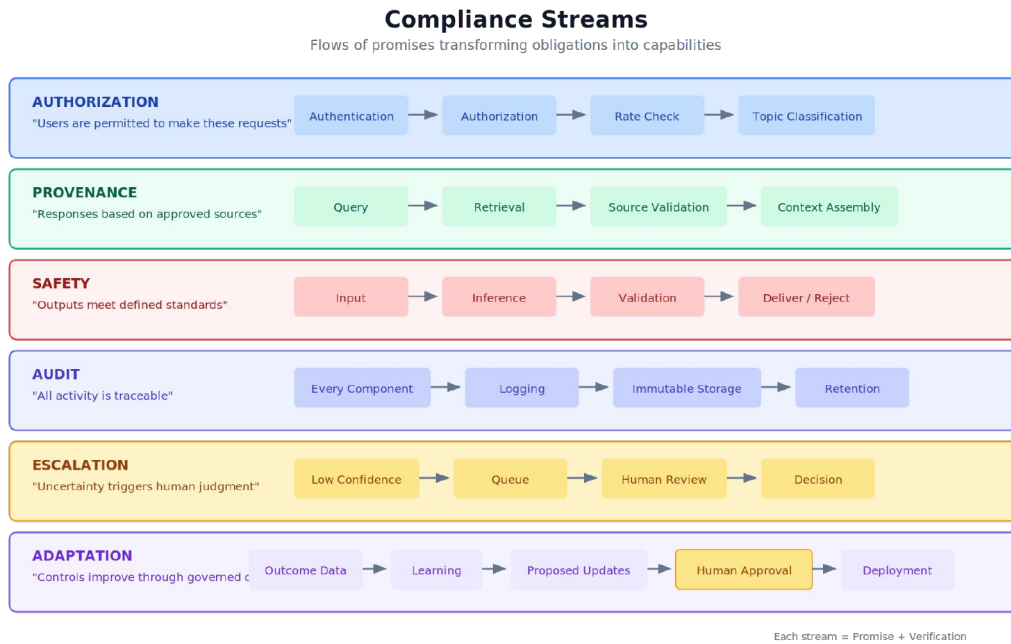
NIST AI RMF — The Govern function sits above Map, Measure, Manage. Governance sets the value context within which technical risk management operates. The framework doesn't assume we'll eventually automate value judgments.

ISO 42001 — Management commitment and organizational context come first. The management system implements values determined at leadership level. Technical controls serve human governance, not the reverse.

None of these frameworks assume technical systems will eventually handle ethical regulation autonomously. They build technical controls beneath permanent human governance — exactly the structure Ashby's analysis predicts.

Compliance Streams: Operational Promises in Flow

Viewing this through a compliance streams lens — flows of promises that transform obligations into operational capabilities — clarifies what operational compliance actually looks like in practice.



Each stream represents a specific operational promise with explicit verification. These aren't policy statements; they're commitments verified on each transaction:

Authorization stream: "This user is permitted to make this request." Flows from authentication through authorization through rate limiting through topic classification. Every request is checked.

Provenance stream: "This response is based on approved information sources." Flows from query through retrieval through source validation through context assembly. Every piece of context is traced.

Safety stream: "This output meets defined standards." Flows from input through inference through validation through delivery or rejection. Every output is verified.

Audit stream: "This activity is traceable and reconstructable." Flows from every component through immutable logging through retention. Every action is recorded.

Escalation stream: "Uncertainty triggers human judgment." Flows from low confidence through queue through human review through decision. Every exception is handled.

Adaptation stream: "Controls improve over time through governed change." Flows from outcome data through learning through proposed updates through human approval through deployment.

The first five streams are first-order operational compliance — verification happening on each transaction. The sixth stream is second-order — ensuring the compliance machinery stays current. Human approval in the adaptation stream is where second-order meets third-order: operational compliance proposes, governance decides.

What This Framework Does and Does Not Solve

What it provides

A integrated operational compliance system. All three orders of regulation — transaction controls, adaptive controls, and governance — integrated within a single framework.

Defense in depth. Multiple control surfaces, each catching what the others miss.

Auditability at every level. Complete traceability of system behavior, control adaptations, and governance decisions.

Adaptability. Learning controls that maintain requisite variety as conditions change.

Integrated escalation. Defined paths through all three orders of regulation, with governance as the third-order layer within the system.

Theoretical grounding. A principled basis for why the architecture is structured as it is and what each layer can and cannot do.

What it does not provide

Guarantees about model behavior. The model internals remain opaque. We govern boundaries, not the transformation itself.

Complete coverage. Operational compliance is probabilistic, not exhaustive. Novel inputs may produce novel failures.

Automated value judgments. Third-order regulation — decisions about what outcomes should be permitted — requires human judgment. The framework integrates this judgment; it doesn't replace it.

Resolution of value conflicts. When safety and helpfulness conflict, when different stakeholders have different values, the architecture provides the machinery to implement and document decisions but not to make them.

The honest position

The compliance posture this architecture supports is not "this system cannot fail" but rather:

We have defined controls at every order of regulation. We verify compliance on every transaction. We adapt controls as conditions change. We make governance decisions through documented, accountable processes. We can demonstrate what happened and why at every level — from individual transactions through control adaptations through the value judgments that set our criteria.

That is a defensible position — rigorous and honest about what each layer contributes.

Practical Implications

For organizations deploying LLMs in regulated environments:

Treat governance as part of operational compliance, not separate from it. Value judgments about acceptable outcomes are the third-order layer of the system. Subject them to the same discipline: documentation, traceability, version control, continuous improvement.

Design for all three orders. Invest in first-order controls (transaction verification), second-order controls (adaptation and drift detection), and third-order controls (governance processes that are themselves auditable).

Accept probabilistic assurance. Operational compliance for LLMs is statistical, not absolute. The posture is "we verify every transaction against defined criteria, adapt as conditions change, and document governance decisions" — not "failures are impossible."

Plan for learning at every level. Static controls become insufficient. First-order controls must adapt to new patterns. Second-order controls must detect drift. Third-order governance must revisit criteria as contexts evolve.

Make escalation a flow within the system. The path from transaction controls through adaptive controls to governance decisions should be a defined compliance stream, not a handoff to something external.

Be transparent about what each order contributes. Stakeholders — including regulators — are better served by clear explanation of the three orders and what each can and cannot guarantee than by vague claims about system safety.

Conclusion

Governing LLMs requires a integrated operational compliance model — one that integrates all three orders of regulation within a single framework.

The system under regulation has variety that exceeds any static control regime. The transformation from input to output is opaque by nature. R. Ashby's Law of Requisite Variety tells us that effective regulation requires matching the variety of the system being regulated — which means learning, adaptive controls rather than static rules.

But M. Ashby's framework also identifies three orders of regulation: first-order controls acting on transactions, second-order controls ensuring those controls remain effective, and third-order regulation involving value judgments about what outcomes should be permitted.

In an integrated model, all three orders are part of operational compliance. Governance is not external — it is the third-order layer within the system. This means governance decisions are subject to the same discipline as technical controls: documented, justified, versioned, auditable, and subject to continuous improvement.

The regulatory frameworks emerging around AI — EU AI Act, NIST AI RMF, ISO 42001 — all require both technical controls and human oversight. An integrated operational compliance model satisfies both: the technical layers provide first and second order regulation; the governance layer provides third-order regulation; and all three are integrated, traceable, and accountable.

That is what this architecture provides. Not a separation of concerns between "technical" and "governance" functions, but an integrated system where every order of regulation operates with appropriate rigor.

The goal is not to eliminate human judgment from AI governance. The goal is to build first and second order controls that make human third-order judgment effective — informed, timely, and actionable. That is the work this integrated operational compliance architecture supports.

About the Author

Raimund Laqua is a Professional Engineer and founder of Lean Compliance Consulting, applying lean principles and systems thinking to regulatory compliance across high-risk industries. He serves on ISO's ESG working group and OSPE's AI in Engineering committee.

References

Ashby, W. Ross. *An Introduction to Cybernetics* (1956)
Ashby, W. Ross. *Design for a Brain* (1952)
Ashby, Mick. "Ethical Regulators and Super-Ethical Systems." *Systems* 8, no. 4 (2020): 53
Ashby, Mick. Ethical Regulator Theorem (2017)

Document Information

Version: 2.0

Date: November 2025

Classification: Public